Data Science: Machine Learning and Natural Language Processing

Reinforcement learning for sequential decision making
Time Series: Random forests and gradient boosting for temporal data

Professor: Patrick Houlihan

Reinforcement learning for sequential decision making

Learning Methods

- The focus of learning is acquiring skills or knowledge from experience.
- As such, there are many different types of learning methods in the field of AI/Machine Learning.
- The three well-known and widely studied learning problems in research and practice are:

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

1. Supervised Learning

Supervised learning describes a class of learning problems that involve using a model to learn a mapping between input examples and the target variable.

Models are fit on training data comprised of inputs and outputs and used to make predictions on test sets where only the inputs are provided and the outputs from the model are compared to the withheld target variables and used to estimate the skill of the model.

There are two main types of supervised learning problems:

Classification: Supervised learning problem that involves predicting a class label.

Regression: Supervised learning problem that involves predicting a numerical value.

Both classification and regression problems may have one or more input variables and input variables may be any data type, such as numerical or categorical.

2. Unsupervised Learning

Unsupervised learning describes a class of learning problems that involve using a model to describe or extract relationships in data.

Compared to supervised learning, unsupervised learning operates upon only the input data without outputs or target variables. As such, unsupervised learning does not have a teacher correcting the model, as in the case of supervised learning.

There are many types of unsupervised learning, although there are two main problems that are often encountered :

Clustering: Unsupervised learning problem that involves finding groups in data.

•**Density Estimation**: Unsupervised learning problem that involves summarizing the distribution of data.

3. Reinforcement Learning

Reinforcement learning describes a class of learning problems where an agent operates in an environment and must *learn* how to operate using feedback.

Reinforcement learning is learning what to do — how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them.

The use of an environment means that there is no fixed training dataset, rather a goal or set of goals that an agent is required to achieve, actions it may perform, and feedback about performance toward the goal.

An example of a reinforcement problem is playing a game where the agent has the goal of getting a high score and can make moves in the game and receive feedback in terms of punishments or rewards.

Supervised vs. Unsupervised vs. Reinforcement Learning



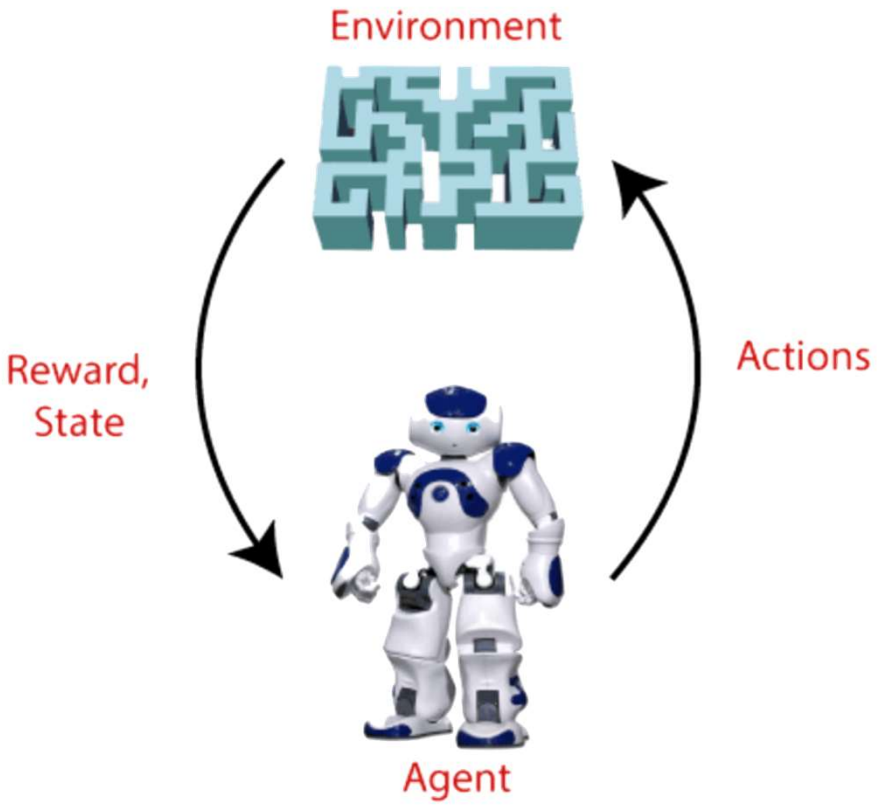




Criteria	Supervised ML	Unsupervised ML	Reinforcement ML
Definition	Learns by using labelled data	Trained using unlabelled data without any guidance.	Works on interacting with the environment
Type of problems	Regression and classification	Association and Clustering	Exploitation or Exploration
Algorithms	Linear Regression, Logistic Regression, SVM, KNN etc.	K – Means, C – Means, Apriori	Q – Learning, SARSA
Aim	Calculate outcomes	Discover underlying patterns	Learn a series of action
Application	Risk Evaluation, Forecast Sales	Recommendation System, Anomaly Detection	Self Driving Cars, Gaming, Healthcare

What is Reinforcement Learning?

- Reinforcement learning is a type of machine learning method where an intelligent agent interacts with the environment and learns to act within that.
- For each good action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.
- RL solves a specific type of problem where decision making is sequential, and the goal is long-term.



- The agent continues doing following three things:
  - **Take action,**
  - **change state/remain in the same state**
  - **get feedback**

By doing these actions, he learns and explores the environment.

Term Used in Reinforcement Learning

Agent → An entity that can perceive/explore the environment and act upon it.

Environment → A situation in which an agent is present or surrounded by.

Action → Actions are the moves taken by an agent within the environment.

State → State is a situation returned by the environment after each action taken by the agent.

Reward → A feedback returned to the agent from the environment to evaluate the action of the agent.

Policy → Policy is a strategy applied by the agent for the next action based on the current state.

Value → It is expected long-term retuned with the discount factor and opposite to the short-term reward.

Elements of Reinforcement Learning

There are four main elements of Reinforcement Learning, which are given below

1. Policy:

A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states.

The policy-based approach has mainly two types of policy:

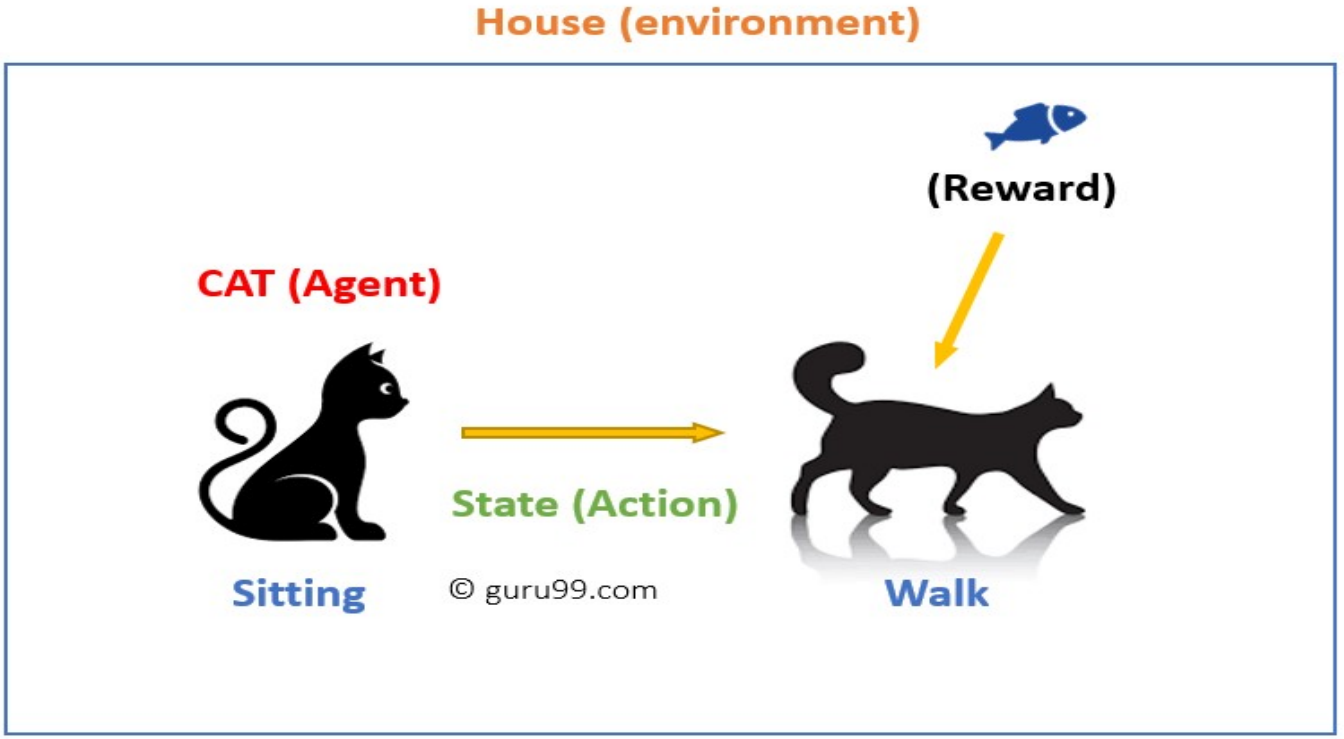
- **Deterministic:** The same action is produced by the policy (π) at any state.
- **Stochastic:** In this policy, probability determines the produced action.

2. Reward Signal: The goal of RL is defined by the reward signal. Reward signals are given according to the good and bad actions taken by the learning agent. The main objective is to maximize the total number of rewards for good actions.

3. Value Function: Gives information about how good the situation and action are and how much reward an agent can expect.

4. Model: Mimics the behavior of the environment.

Reinforcement Learning Example



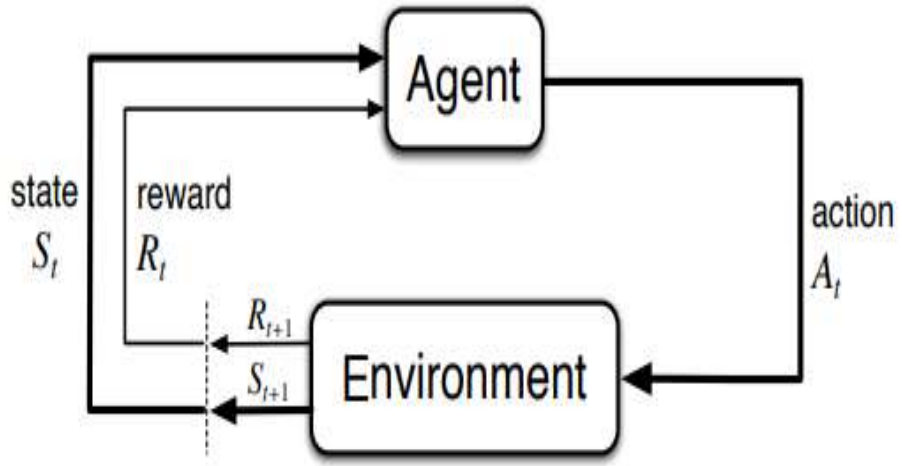
- An example of a state could be your cat sitting, and you use a specific word for cat to walk.
- Your cat reacts by performing an action transition from one "state" (sitting) to another "state" (walking).
- The reaction of your cat is an action, and the policy is a method of selecting an action given a state in expectation of better outcomes.
- After the transition, cat may get a reward or penalty in return.

Reinforcement Learning Applications



In a typical Reinforcement Learning (RL) problem, there is a learner and a decision maker called **agent** and the surrounding with which it interacts is called **environment**. The environment, in return, provides **rewards** and a **new state** based on the **actions** of the agent.

So, in reinforcement learning, we do not teach an agent how it should do something but present it with rewards whether positive or negative based on its actions.



The Agent-Environment Relationship:

Agent : Software programs that make intelligent decisions and they are the learners in RL. These agents interact with the environment by actions and receive rewards based on these actions.

Environment: It is the demonstration of the problem to be solved. We can have a real-world environment or a simulated environment with which our agent will interact.

State : This is the position of the agents at a specific time-step in the environment. So, whenever an agent performs an action, the environment gives the agent a reward and a new state where the agent reached by performing the action.

Transition : Moving from one state to another is called Transition (or state transition).

Transition Probability: The probability that the agent will move from one state, *s* to another state *s'* is called transition probability, $P_{ss'}$.

*The **Markov Property** states that :*

"Future is Independent of the past given the present"

Mathematically we can express this statement as :

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \ldots, S_t]$$

S_t denotes the current state of the agent and S_{t+1} denotes the next state. What this equation means is that the transition from state S_t to S_{t+1} is entirely independent of the past. So, the RHS of the Equation means the same as LHS if the system has a Markov Property. Intuitively meaning that our current state already captures the information of the past states.

State Transition Probability :

As we now know about transition probability we can define state transition probability as follows :

For *Markov States* from S_t to S_{t+1} i.e. any other successor state , the state transition probability is given by,

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

We can formulate the state transition probability into a state transition probability matrix by :

$$P = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ & \cdots & \cdots & \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{bmatrix}$$

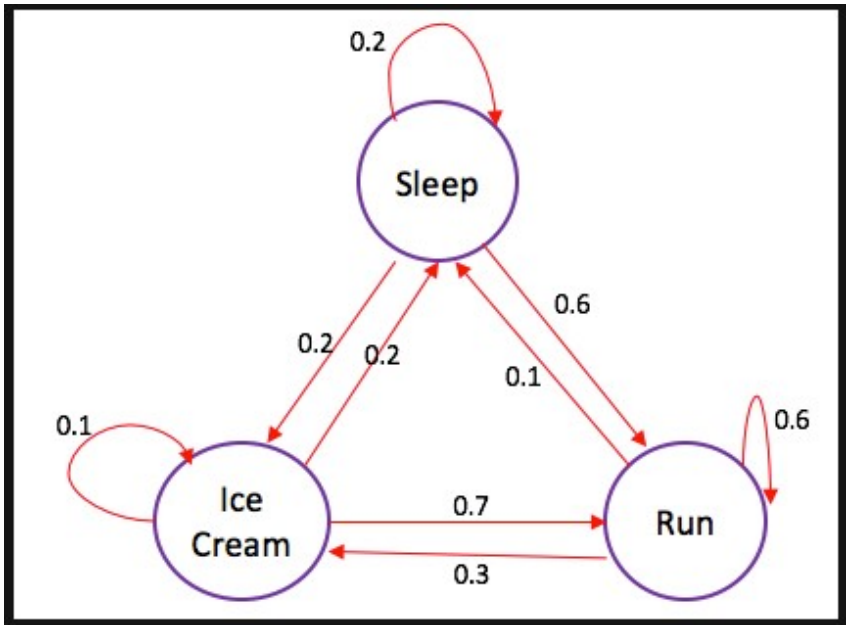
Each row in the matrix represents the probability from moving a particular state (starting state) to any successor state. Sum of each row is equal to 1.

Markov Process or Markov Chains

A *Markov Process* is the memoryless **random process** i.e. a sequence of random states S_1, S_2,….S_n with a Markov Property. So, it's basically a sequence of states with the Markov Property. It can be defined using a set of states(S) and transition probability matrix (P).The dynamics of the environment can be fully defined using the States (S) and Transition Probability matrix (P).

But what **random process** means ?

To answer this question let's look at an example:



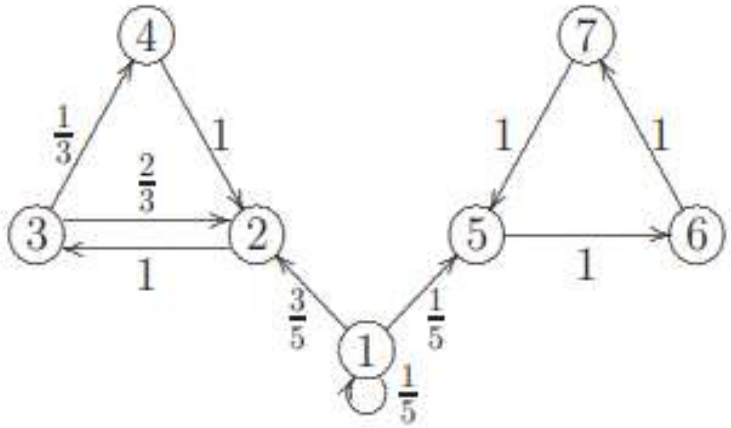
The edges of the tree denote **transition probabilities**. Now, suppose that we were sleeping and the according to the probability distribution there is a **0.6** chance that we will **Run** and **0.2** chance we **sleep more** and again **0.2** that we will eat **ice-cream**.

Now, we can take any state sequence from this MDP and compute the corresponding probability of this state chain as follows:

Given the state sequence $X_0 \to X_1 \to X_2 \to \ldots \to X_k$, the probability of $P(X_0, X_1, X_2, \ldots, X_k)$ is defined as

$$P(X_0, X_1, X_2, \ldots, X_k) = P(X_0) \times P_{01} \times P_{12} \times \ldots \times P_{(k-1)k}.$$

For Example:



Let $P(X_0) \sim (3/4, 0, 1/4, 0, 0, 0, 0)$. What is the probability of the state trajectory (or path) 1, 2, 3,

$$\begin{aligned}
\mathbb{P}(1,2,3,2,3,4) &= \mathbb{P}(X_0 = 1) \times p_{12} \times p_{23} \times p_{32} \times p_{23} \times p_{34} \\
&= \tfrac{3}{4} \times \tfrac{3}{5} \times 1 \times \tfrac{2}{3} \times 1 \times \tfrac{1}{3} \\
&= \tfrac{1}{10}.
\end{aligned}$$

Rewards and Returns

Rewards are the **numerical values** that the agent receives on performing some action at some **state (s)** in the environment. The numerical value can be positive or negative based on the actions of the agent.

In **Reinforcement learning**, we care about **maximizing** the cumulative reward (all the rewards agent receives from the environment) instead of, the reward agent receives from the current state (also called *immediate reward*).

This **total sum of reward** the agent receives from the environment is called **returns**.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \ldots\ldots + R_T$$

where: R[t+1] is the reward received by the agent at time step t[0] while performing an action (a) to move from one state to another. Similarly, R[t+2] is the reward received by the agent at time step t[1] by performing an action to move to another state. And, R[T] is the reward received by the agent by at the final time step by performing an action to move to another state.

Episodic and Continuous Tasks

Episodic Tasks*: These are the tasks that have a* ***terminal state*** *(end state). We can say that they have finite states. For example, in racing games, we start the game (start the race) and play it until the game is over (race ends!). This is called an episode. Once we restart the game it will start from an initial state and hence, every* ***episode*** *is independent.*

Continuous Tasks *: These are the tasks that have no ends i.e. they* ***don't have any terminal state****. These types of tasks will* ***never end****. For example, Learning how to code!*

Now, it's easy to calculate the returns from the episodic tasks as they will eventually end but what about continuous tasks, as it will go on and on forever. The returns sum up to infinity! So, how we define returns for continuous tasks?

This is where we need **Discount factor(ɤ)**.

Discount Factor (ɤ): It determines how much **importance** is to be given to the **immediate reward and future rewards**. This basically helps us to avoid **infinity** as a reward in continuous tasks. It has a value between 0 and 1. A value of **0** means that more importance is given to the **immediate reward** and a value of **1** means that more importance is given to **future rewards**.

In practice, a discount factor of 0 will never learn as it only considers immediate reward and a discount factor of 1 will go on for future rewards which may lead to infinity. Therefore, **the optimal value for the discount factor lies between 0.2 to 0.8**.

So, we can define returns using discount factor as follows :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

Markov Reward Process

Till now we have seen how a Markov chain defined the dynamics of an environment using set of states (S) and Transition Probability Matrix (P). But, we know that Reinforcement Learning is all about the goal to maximize the reward. So, let's ***add reward*** to our Markov Chain. This gives us **Markov Reward Process.**

Markov Reward Process (MRP): As the name suggests, MRPs are the Markov chains with values judgement. Basically, we get a value from every state our agent is in.

Mathematically, we define Markov Reward Process as :

$$R_s = E[R_{t+1} \mid S_t]$$

What this equation means is how much reward (R_s) we get from a particular state S[t]. This tells us the *immediate reward* from that particular state our agent is in. We will see how we maximize these rewards from each state our agent is in; that is maximizing the cumulative reward we get from each state.

We define a Markov Reward Process (MRP) (S,P,R,γ) as follows:

Where:

- S is a set of states,
- P is the Transition Probability Matrix,
- R is the Reward function,
- γ is the discount factor.

Markov Decision Processes: Policy and Value Functions

A *Value Function* determines how good it is for the agent to be in a particular state s. Of course, to determine how good it will be to be in a particular state, it must depend on some actions that it will take. This is where policy comes in.

A policy defines what actions to perform in a particular state **s**.

A policy is a simple function, that defines a **probability distribution** over Actions (a∈ A) for each state (s ∈ S).

If an agent at time t follows a policy π then **π(a|s)** is the probability that agent takes action *(a)* at particular time step *(t)* in state *s*. In Reinforcement Learning, the experience of the agent determines the change in policy.

Mathematically, a policy is defined as follows :

$$\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$$

Now, **how we find a value of a state**. The value of state s, when agent is following a policy π, which is denoted by $V_\pi(s)$, is the expected return starting from state **s** and following a policy π for the next states, until we reach the terminal state. We can formulate this as (**This function is also called State-value Function**):

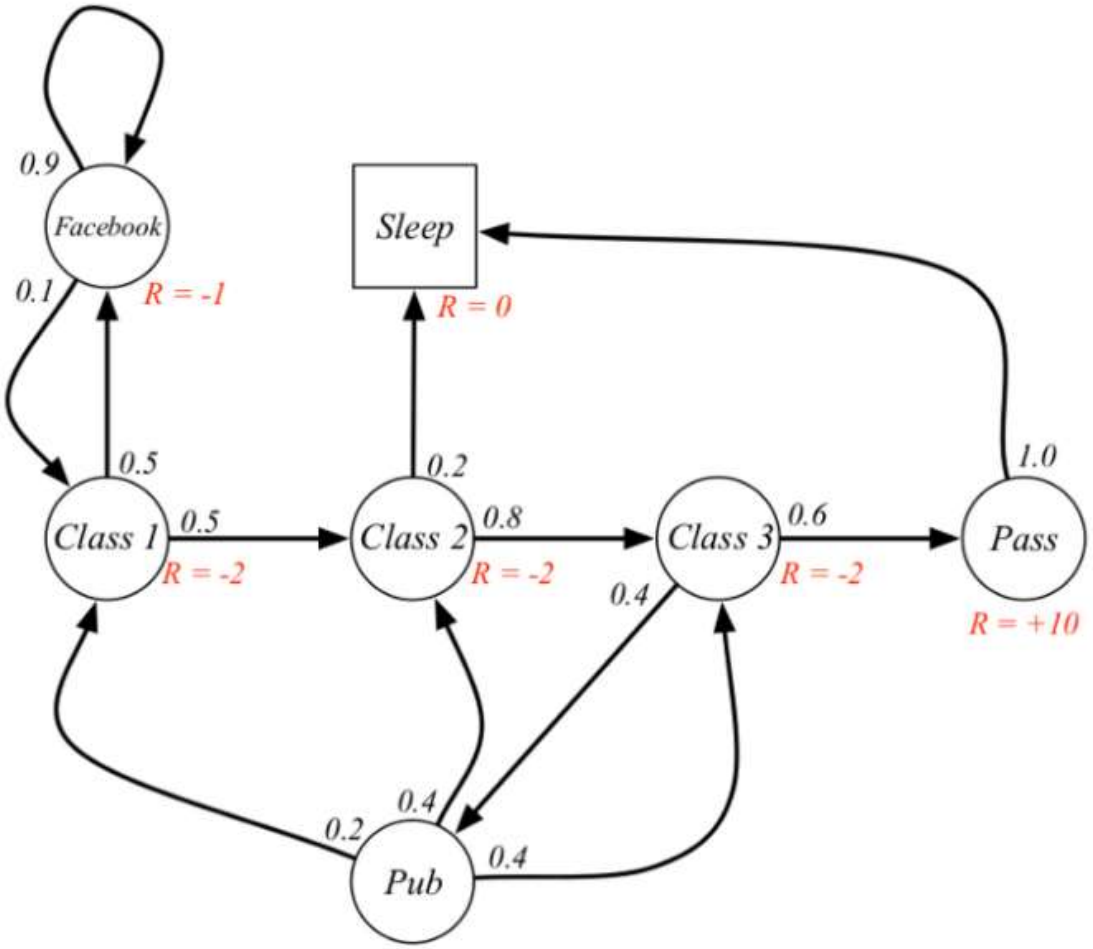
$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \text{ for all } s \in \mathcal{S},$$

This equation gives us the **expected returns** starting from **state(s)** and going to successor states thereafter, with the policy π. Note that the value of a terminal state (if there is any) is zero.

Example: Suppose our start state is *Class 2*, and we follow the state sequence: *Class 2* > *Class 3* > *Pass* > *Sleep*. The *expected return* with discount factor 0.5 is:

-2 + (-2 * 0.5) + 10 * 0.25 + 0 = -0.5

Hence the return of state *Class 2* for this sequence state transitions (moves) is -0.5.



Bellman Equation for Value Function

Bellman Equation helps us to find **optimal policies** and **value functions**. We know that our policy changes with experience so we will have different value functions according to different policies.

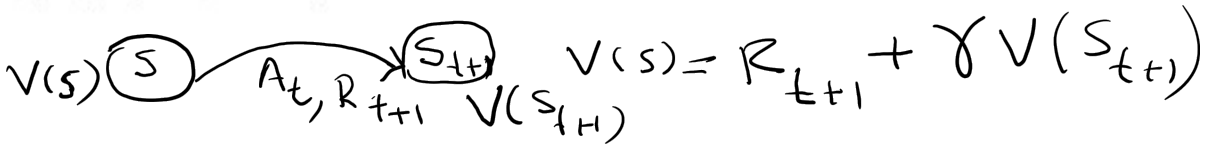
Optimal value function is one which gives maximum value compared to all other value functions.

Bellman Equation states that value function can be **decomposed** into two parts: Immediate Reward, R[t+1] and Discounted value of successor states.

Mathematically, we can define Bellman Equation as :

$$v(s) = \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]$$

We want to know the **value** of state s. The value of state *(s)* is the reward we got upon being in that state, plus the discounted value of the state we landed upon.



What is a Markov Decision Process (MDP) ?

Markov Decision Process *: It is Markov Reward Process with decisions. Everything is same like MRP but now we have actual agency that makes decisions or take actions.*

It is a tuple of ($\boldsymbol{S}$, $\boldsymbol{A}$, $\boldsymbol{P}$, $\boldsymbol{R}$, $\boldsymbol{\gamma}$) where:

- S is a set of states,
- A is the set of actions agent can choose to take,
- P is the transition Probability Matrix,
- R is the Reward accumulated by the actions of the agent,
- $\boldsymbol{\gamma}$ is the discount factor.

P and R will have slight changes w.r.t actions as follows :

Transition Probability:

$$\mathcal{P}^{a}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$

Reward Function:

$$\mathcal{R}^{a}_{s} = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$$

Now, our reward function is dependent on the action. Till now we have talked about getting a reward (R) when our agent goes through a set of states (s) following a policy π. Actually, in Markov Decision Process(MDP) the policy is the mechanism to take decisions . So now we have a mechanism which will choose to take an action.

Policies in an MDP depends on the current state. They do not depend on the history. That's the Markov Property. So, the current state we are in characterizes the history.

We have already seen how good it is for the agent to be in a particular state (State-value function). Now, let's see how good it is to take a particular action following a policy π from state s (Action-Value Function).

The Bellman Expectation equation under a policy π can be stated as:

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]$$

The above equation tells us that the value of a particular state is determined by the immediate reward plus the value of successor states when we are following a certain policy (**π**)**.**

State-action value function or Q-Function:

This function specifies the **how good** it is for the agent to take action (a) in a state (s) with a **policy π**. This function can be mathematically expressed as:

$$q_\pi(s,a) \doteq \mathbb{E}_\pi[G_t \mid S_t=s, A_t=a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \,\middle|\, S_t=s, A_t=a\right]$$

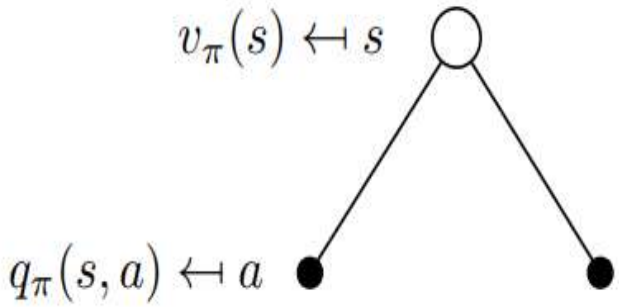
Basically, it tells us the value of performing a certain action(a) in a state(s) with a policy π.

Similar to $V_\pi(s)$, we can express our state-action Value function (Q-Function) as follows :

$$q_\pi(s,a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

From the above equation, we can see that the State-Action Value of a state can be decomposed into the **immediate reward** we get on performing a certain action in state(**s**) and moving to another state(**s'**) plus the discounted value of the state-action value of the state(**s'**) **with respect to** the some action(**a**) our agent will take from that state on-wards.

Now, let's go deeper into Bellman Expextation Equation:
Consider the diagram on the right. From the state s, we
Take several actions with some probability distributio
There is a Q-value(State-action value function) for ea
of the action. Now, we can average the Q-values whi
tells us how good it is to be in a particular state.
Basically, it defines V_π**(s)**

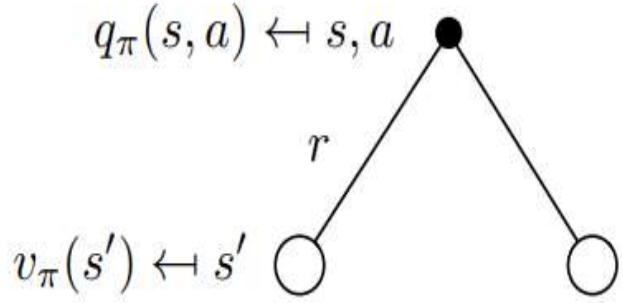


$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a)$$

This equation also tells us the connection between State-Value function and State-Action Value Function.

Now, let's look at the following diagram for State-Action Value Function:

This diagram says that suppose we start off by taking some action (a). So, because of the action (a) the agent might move into any of neigbour states in the environment. Now: ***how good it is to take action(a)?***



We again average the state-values of neighbour states, added with an immediate reward which tells us how good it is to take a particular action (a).This defines the q_π**(s,a).**

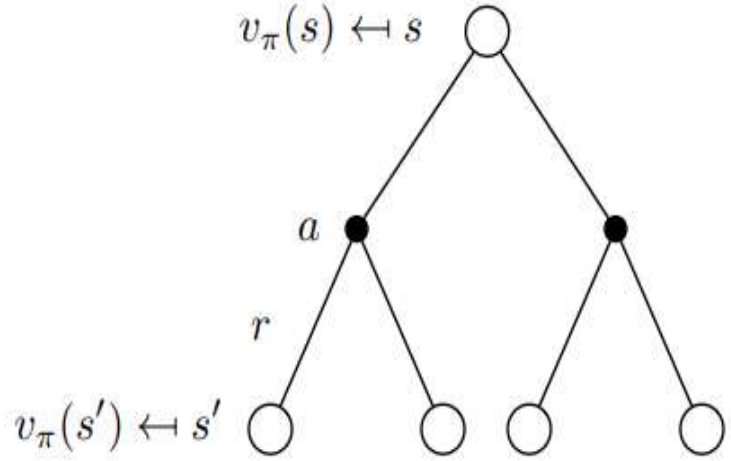
Mathematically, this can be expressed as follows :

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

where P is the Transition Probability Matrix .

Now let's stitch these backup diagrams together to define State-Value Function, V**π(s)**:

Note that the probability of the action the agent might take from state **s** is weighted by our policy and after taking that action the probability that we land in any of the states(**s'**) is weighted by the environment.



Now our question is, how good it is to be in state(s) after taking some action and landing on another state(s') and following our policy($\boldsymbol{\pi}$) after that?

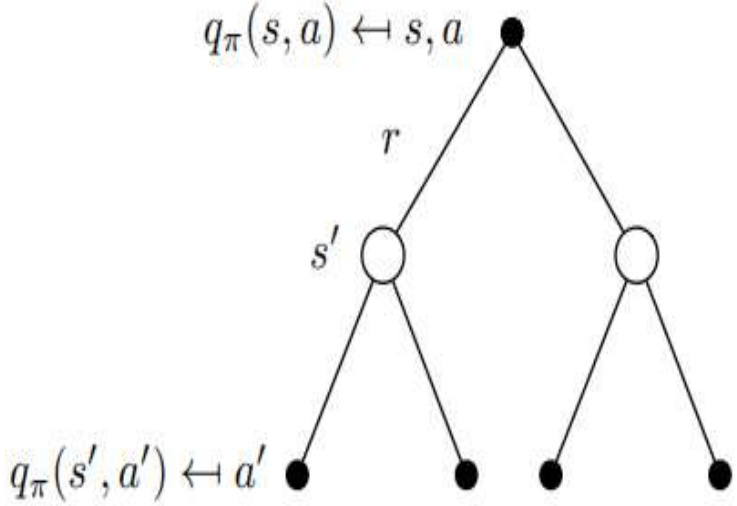
It is similar to what we have done before, we are going to average the value of successor states(**s'**) with some transition probability(P) weighted with our policy.

Mathematically, we can define it as follows:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

Now, let's do the same for State-Action Value Function, q_π**(s,a)** :

It's very similar to what we did in **State-Value Function**, so this diagram basically says that our agent take some action (**a**) because of which the environment might land us on any of the states (**s'**), then from that state we can choose to take any actions (**a'**) weighted with the probability of our policy (**π**). Again, we average them together and that gives us how good it is to take a particular action following a particular policy (**π**) all along.



Mathematically, this can be expressed as :

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s', a')$$

So, this is how we can formulate Bellman Expectation Equation for a given MDP to find it's State-Value Function and State-Action Value Function.

But, it does not tell us the best way to behave in an MDP.

For that let's talk about what is meant by **Optimal Value** and **Optimal Policy Function**.

Optimal Value Function

In an MDP environment, there are many different value functions according to different policies. ***The optimal Value function is one which yields maximum value compared to all other value function***. When we say we are solving an MDP it actually means we are finding the Optimal Value Function.

So, mathematically Optimal State-Value Function can be expressed as :

$$v_*(s) = \max_{\pi} v_\pi(s)$$

In the above formula, v∗(s) tells us what is the maximum reward we can get from the system.

Defining Optimal State-Action Value Function (Q-Function)

Similarly, **Optimal State-Action Value Function** tells us the maximum reward we are going to get if we are in state *s* and taking action *a* from there on-wards.

Mathematically, It can be defined as :

$$q_*(s, a) = \max_{\pi} q_\pi(s, a)$$

Optimal State-Value Function *:It is the maximum Value function over all policies.*

Optimal State-Action Value Function*: It is the maximum action-value function over all policies.*

Optimal Policy

Before we define Optimal Policy, let's know, ***what is meant by one policy better than other policy?***

We know that for any MDP, there is a policy (**π)** better than any other policy (**π'). But How?**

We say that one policy (**π)** is better than other policy (**π'**) if the value function with the policy **π** for all states is greater than the value function with the policy **π'** for all states. Intuitively, it can be expressed as :

$$\pi \geq \pi' \text{ if } v_{\pi}(s) \geq v_{\pi'}(s), \forall s$$

Now, let's define ***Optimal Policy :***

Optimal Policy *is one which results in optimal value function.*

Note that, *there can be more than one optimal policy* in a MDP. But, ***all optimal policies achieve the same optimal value function and optimal state-action Value Function(Q-function)***.

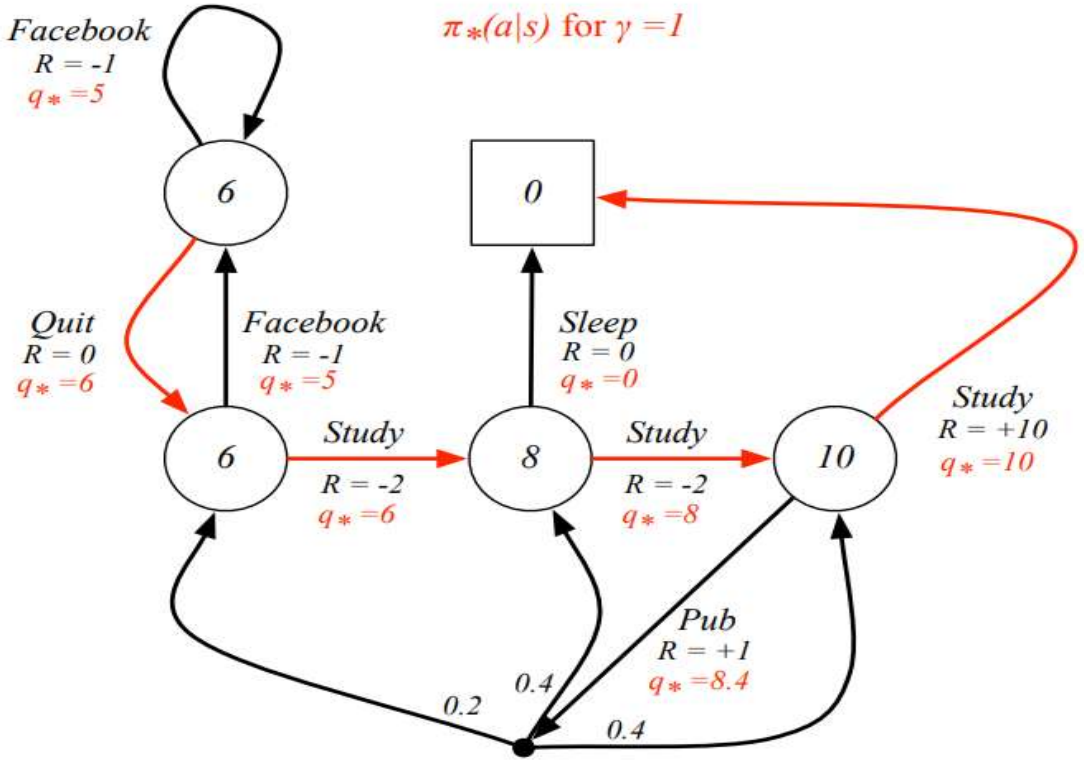
Now, the question arises how we find Optimal Policy.

We find an optimal policy by maximizing over ***q****(s, a) i.e. our optimal state-action value function. We solve q*(s,a) and then we pick the action that gives us most optimal state-action value function (q*(s,a)). That is:

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s,a) \\ 0 & \textit{otherwise} \end{cases}$$

That is, for a state s we pick the action a with probability 1, if it gives us the maximum q*(s,a). So, if we know q*(s,a) we can get an optimal policy from it.

Let's consider the following example:



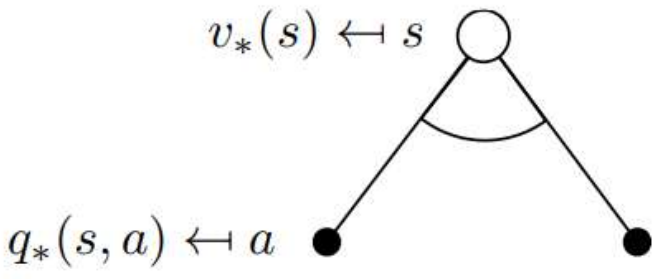
In this example, the red arcs are the **optimal policies** which means that if our agent follows this path it will **yield maximum reward** from this MDP. Also, by seeing the **q*** values for each state we can say the actions our agent will take that yields maximum reward. So, optimal policy always takes action with higher q* value (State-Action Value Function). For example, in the state with value 8, there is q* with value 0 and 8. Our agent chooses the one with greater **q*** value i.e. 8.

Now, the question arises, ***How do we find these q*(s,a) values ?***

This is where Bellman Optimality Equation comes into play.

Bellman Optimality equation is the same as Bellman Expectation Equation but the only difference is instead of taking the average of the actions our agent can take the action with the max value.

Let's look at the Backup diagram:

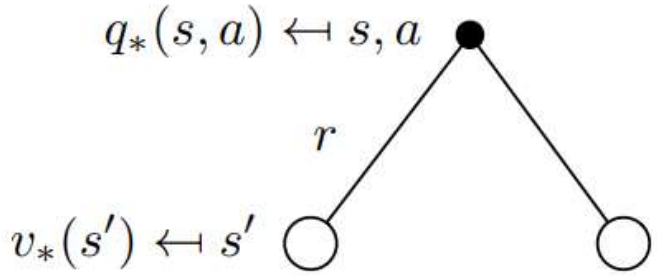


Suppose our agent is in state s and from that state it can take two actions. So, we look at the action-values for each of the actions and our agent takes the action with **greater q* value**. This gives us the value of being in the state s.

Hence, mathematically, the Bellman optimality equation for state-value function is

$$v_*(s) = \max_a q_*(s, a)$$

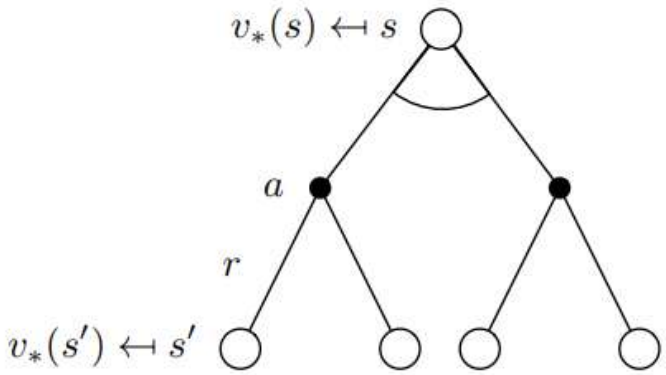
Similarly, let's define Bellman Optimality Equation for **State-Action Value Function (Q-Function)**:



Suppose, our agent has taken an action a in some state s. We still take the average of the values of both the states, but the only **difference** is in Bellman Optimality Equation, we know the **optimal values** of each of the states:

$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Let's again stitch these backup diagrams for State-Value Function :

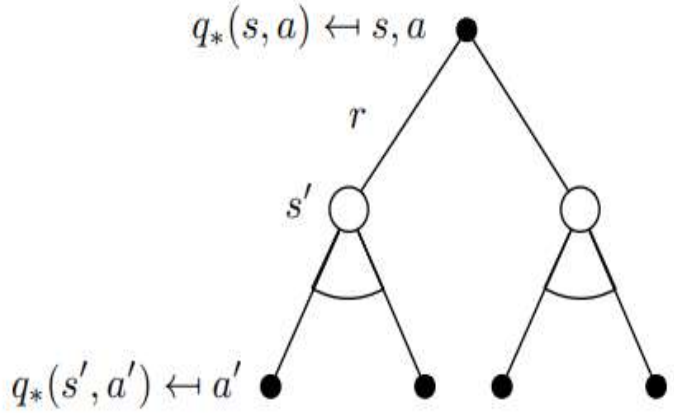


Suppose our agent is in state s and from that state it took some action (a) where the probability of taking that action is w**eighted** by the policy. And because of the action (a), the agent might get blown to any of the states(**s'**) where probability is weighted by the environment. **In order to find the value of state s we simply average the Optimal values of the states (s')**. This gives us the value of being in state s.

The max in the equation is because we are maximizing the actions the agent can take in the upper arcs. This equation also shows how we can relate V* function to itself.

$$\Longleftrightarrow \quad v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in S} \mathcal{P}_{ss'}^a v_*(s')$$

Now, let's look at the Bellman Optimality Equation for State-Action Value Function, q*(s,a) :

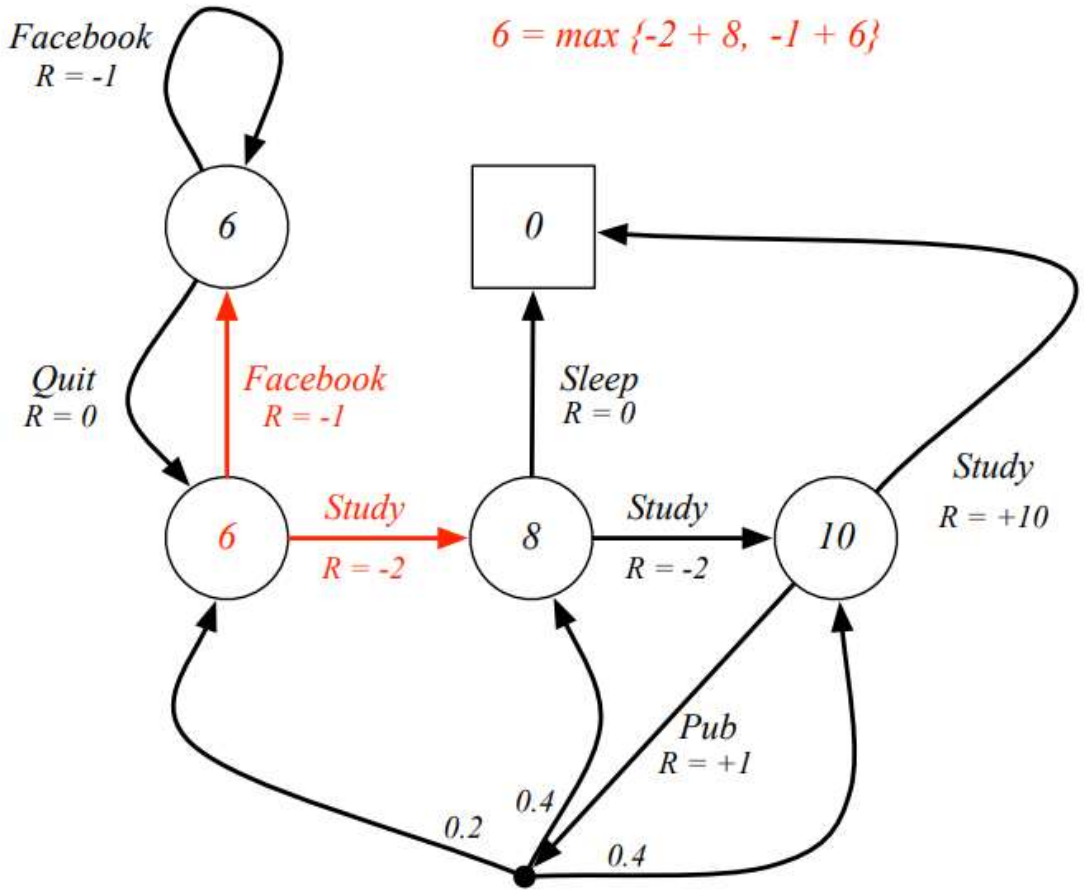


Suppose, our agent was in state **s** and it took some action (**a**). Because of that action, the environment might land our agent to any of the states (**s'**) and from these states we get to **maximize** the action our agent will take i.e. choosing the action with **maximum q* value**. We back that up to the top and that tells us the value of the action a.

Mathematically, this can be expressed as :

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a')$$

Let’s look at the following example to understand it better :



Look at the red arrows, suppose we wish to find the **value** of state with value 6 (**in red**), as we can see we get a reward of -1 if our agent chooses Facebook and a reward of -2 if our agent choose to study. In order to find the value of state in red, we will use the **Bellman Optimality Equation for State-Value Function** i.e. *considering the other two states have optimal value we are going to take an average and maximize for both the action (choose the one that gives maximum value)*. So, from the diagram we can see that going to Facebook yields a value of 5 for our red state and going to study yields a value of 6 and then we maximize over the two which gives us 6 as the answer.

Now, how do we solve Bellman Optimality Equation for large MDPs. In order to do so we use **Dynamic Programming algorithms** like **Policy iteration** and Value iteration.

What is Dynamic Programming?

Dynamic Programming is a lot like divide and conquer approach which is breaking down a problem into sub-problems but the only difference is instead of solving them independently (like in divide and conquer), results of a sub-problem are used in similar sub-problems.

A problem can be solved using dynamic programming if it satisfies two properties:

•**Optimal Substructure**: This means that a problem can be divided into sub-problems and if we find optimal solutions to those sub-problems, then we can use this optimal solution to find an optimal solution for the overall problem.

•**Overlapping Sub-problems**: Being said before, Dynamic Programming is used if a problem has similar sub-problems. The solution of one sub-problem is saved and then used to solve similar sub-problems.

Do Markov Decision Process have these two properties?

For the first property, recall, Bellman equation is defined as follows:

$$v(s) = \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]$$

Remember what the Bellman Equation said: the value of a state is equal to the immediate reward our agent gets leaving the state **plus** the value of the next state. So, the equation is **breaking down** the process of **finding the value function** of a state by dividing it into sub-problems.

For overlapping sub-problems, we have Value functions. ***V***alue functions have already stored how good a particular state is so we don't need to recompute the value of that state again and again. ***For example, suppose there are two*** states(s1 and s2) and we are in state s1 and we have already computed the value of state s2 so when calculating the value of state s1 we will not re-compute the value of state s2. (The value of s1 relies on state s2)

What we mean when we say we are going to solve a Markov Decision Process?

It means that we are going to find the Value Function that will satisfy the Bellman Equation. Once we have found the optimal value function, then we can use it to find the optimal policy.

This process is broken down into two parts:

1.Prediction: Suppose we have a Markov Decision Process defined as (S, A, P, R, γ) and given some policy π. Our job in prediction is to find the value function. We find the value function by evaluating a policy using the Bellman Expectation Equation.

2.Control: This process involves optimizing the value function, we calculated during the prediction process. We find **optimal value function** and **optimal policy** for our ***Markov Decision Process***.

Policy Evaluation

Synchronous Backups: *It means that we are going to take into consideration every state in our Markov Decision Process for calculating the value of a state.*

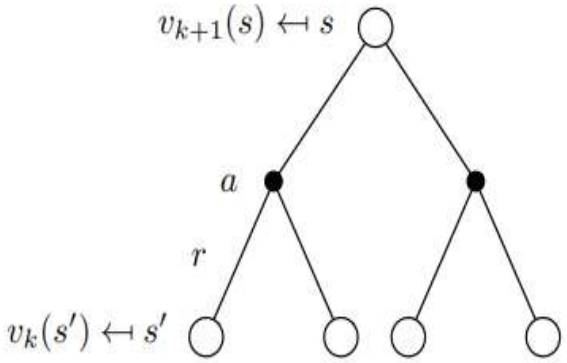
Policy Evaluation means how much reward our agent is going to get by following a particular policy π. ***But how we evaluate a policy π?***

We evaluate a given policy π using Bellman Expectation Equation. Recall that, Bellman Expectation Equation tells how much reward our agent is going to get following a policy π and is defined as:

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

Initially we start with our value function of each state initialize to zero.

Now, to evaluate a policy we are going to use this equation in an iterative fashion. Meaning, we will calculate the value of the next state by backing up the value of the current state from the previous iteration. Lets understand this with an example:



We know that the value of the root node [s] is given by the reward we got by taking the action [a] plus the value of the state [s']. Now, what we are doing is we are using the Bellman Expectation equation in an iterative fashion for dynamic programming. That is, we will use the value of the leaves (state [s'] here) which is from the previous iteration, when we were computing the value of state s, to calculate the value of the state [s]. This happens in every iteration. Now, for the second iteration, we are going to use the value of state we computed in the previous iteration that is v [k+1] to compute the value of the next state, that is, v [k+2].

Random Forest for Time Series Forecasting

Applying Machine Learning to Temporal Data

Using Random Forest algorithms for predicting future values in time series data

What is Random Forest?

Random Forest is an ensemble machine learning algorithm combining multiple decision trees.

Key Characteristics:

- **Ensemble Method:** Builds multiple decision trees and merges them for more accurate predictions
- **Bootstrap Aggregating (Bagging):** Trains each tree on random subsets of data
- **Versatile:** Works for both classification and regression problems
- **Robust:** Lower variance than individual decision trees, resistant to overfitting

The Challenge: Time Series Data

Why Time Series is Different:

- **Temporal Dependencies:** Observations are NOT independent and identically distributed
- **Serial Correlation:** Past values influence future values
- **Time Ordering:** The sequence of data points matters critically

Key Problem: Random Forest algorithms assume data independence, which is violated in time series where observations have temporal relationships.

Important Limitation

Random Forests Cannot Extrapolate

Tree-based models operate using if-then rules that split the input space recursively.

> **Critical Constraint:** Random Forests can only predict values within the range of the training data target variable. They cannot predict trends outside this range.

Implication:

- Models must be retrained frequently as new data arrives
- Trending data requires special preprocessing (differencing)
- Linear extrapolation is not possible without transformation

Transforming Time Series Data

To use Random Forest, we must convert time series into a supervised learning problem.

Sliding Window Approach:

- **Create Lag Features:** Use past values as predictor variables (e.g., t-1, t-2, t-3)
- **Define Target:** Future value(s) to predict (e.g., t+1)
- **Generate Training Examples:** Each row becomes an input-output pair

Example: To predict tomorrow's sales, use the past 7 days as features

Essential Preprocessing Steps

1. Handle Missing Values

- Impute or remove
- Required for RF models

2. Remove Trends

- Use differencing
- Apply transformations

3. Create Features

- Lag variables
- Seasonality indicators

4. Scale if Needed

- Normalize features
- Handle outliers

Walk-Forward Validation

Standard k-fold cross-validation will produce optimistically biased results for time series.

Proper Evaluation Method:

- **Preserve Time Order:** Training data always comes before test data
- **Expanding Window:** Train on historical data, test on next time period
- **Iterate Forward:** Move the window forward through time
- **One-Step or Multi-Step:** Predict one or multiple future periods

> This method simulates real-world forecasting where you only have past data to predict the future.

Implementation Workflow

1. **Transform to Supervised Learning:** Create lagged features and target variable
2. **Split Data:** Use walk-forward validation approach
3. **Train Model:** Fit RandomForestRegressor on training set
4. **Make Predictions:** Use trained model to forecast test period
5. **Evaluate Performance:** Calculate MAE, RMSE, or other metrics
6. **Retrain Regularly:** Update model as new data becomes available

Pros and Cons

Advantages ✓

- Handles multivariate data well
- Robust to overfitting
- Captures non-linear patterns
- Feature importance analysis
- Less hyperparameter tuning needed

Disadvantages ✗

- Cannot extrapolate trends
- Requires data transformation
- Computationally intensive
- Less interpretable than ARIMA
- Needs frequent retraining

Key Takeaways

- **Transformation is Essential:** Convert time series to supervised learning format using sliding windows
- **Use Proper Validation:** Walk-forward validation, not k-fold cross-validation
- **Preprocess Carefully:** Remove trends through differencing before modeling
- **Combine with Other Methods:** Consider ensembling with traditional time series models
- **Regular Updates:** Retrain models frequently as new data arrives

Bottom Line: Random Forest can be effective for time series forecasting when properly preprocessed and evaluated, especially for complex multivariate problems.